🌬️ Przetwarzanie strumieniowe — Structured Streaming i Apache Kafka

Dane nie czekają na przerwę świąteczną! 🎄⏩🌊

Python w Zastosowaniach Big Data | Wykład 12

Prowadzący: Pavlo Melnyk

👋 Witamy z powrotem po przerwie świątecznej!

Ostatnie zajęcia (Wykład 11 — Cloud Computing i GCP) odbyły się **15.12.2026**. Nie było zajęć **22.12** ani **29.12.2026** — czas na odpoczynek i świąteczne pierniczki 🍪.

Dziś wracamy z **ostatnim wykładem merytorycznym** kursu — i to porządnym: **Kafka + Spark Structured Streaming**! 🚀

> 🎯 **Po tym wykładzie:** Zrozumiesz jak Spark przetwarza dane w czasie rzeczywistym oraz jak wpasowuje się w to Apache Kafka.

📋 Plan wykładu

1. Batch vs Streaming — przypomnienie i pogłębienie
2. Apache Kafka — koncepcje: topic, partition, producer, consumer, broker
3. Spark Structured Streaming — model unbounded table
4. `readStream` / `writeStream` — podstawowe API
5. Windowed aggregations — okna czasowe
6. Watermarking — obsługa spóźnionych danych
7. Output modes: append / update / complete
8. Architektura Kafka + Spark — jak to działa razem
9. Częste pułapki
10. Podsumowanie całego kursu 🎓
11. Egzamin i projekt — co dalej
12. Q&A

💡 Hook — Czujniki nie robią sobie przerwy świątecznej

Miejski system monitoringu jakości powietrza ma **~30 czujników** rozsianych po mieście. One mierzą PM2.5 **teraz**, **cały czas**, bez przerwy.

```
# Batch — sprawdzimy jutro rano 🐌
raport_wczorajszy = spark.read.parquet("powietrze/2027-01-11/")
alerty = raport_wczorajszy.filter("pm25 > 100")
# Mieszkańcy oddychali smogiem całą noc, zanim ktoś zareagował 😷

# Stream — reagujemy W MOMENCIE przekroczenia normy ⚡
strumien = spark.readStream.format("kafka").load()
alerty_live = strumien.filter("pm25 > 100")
# Alert na telefon w ciągu sekund! 📱🚨
```

> Smog nie czeka na batch job o 3:00 w nocy. Witajcie ponownie w świecie **streamingu**! 🌊

🔄 Batch vs Streaming — szybkie przypomnienie

Cecha	Batch 📦	Streaming 🌊
Dane	Skończony zbiór	Nieskończony strumień
Opóźnienie	Minuty–godziny	Milisekundy–sekundy
Przykład	Miesięczny raport jakości powietrza	Alert o przekroczeniu normy PM2.5
Złożoność	Niższa	Wyższa ⚠️

> 💡 **Analogia:** Batch = comiesięczny przegląd samochodu. Streaming = czujnik ciśnienia w oponach, który ostrzega NATYCHMIAST 🚗💨

🤔 Think about it

Nasze miasto chce wiedzieć o przekroczeniu normy PM2.5 **zanim** poziom stanie się niebezpieczny dla astmatyków. Czy wystarczy batch co godzinę? A co jeśli poziom smogu skacze w 10 minut? 🤔

🔌 Apache Kafka — co to takiego?

Apache Kafka = rozproszony, wysoce skalowalny **message broker** — "poczta" dla danych strumieniowych.

- 📬 **Producent** (producer) wysyła wiadomości (np. czujnik wysyła odczyt PM2.5)
- 📭 **Konsument** (consumer) odbiera i przetwarza wiadomości (np. Spark)
- 🗂️ Wiadomości grupowane są w **topiki** (topics) — np. `powietrze-pm25`
- 🧩 Każdy topik dzieli się na **partycje** (partitions) — równoległość i skalowanie
- 🖥️ **Broker** = serwer Kafki, który przechowuje i serwuje dane; **klaster** = wiele brokerów razem

> 💡 **Analogia:** Kafka to skrzynka pocztowa 📮 miasta — każdy czujnik wrzuca list (odczyt), a Spark codziennie (a właściwie bez przerwy!) odbiera pocztę i ją czyta.

Kafka — Topics i Partitions

```
Topic: "powietrze-pm25"
┌──────────────────────────────────────┐
│ Partition 0: [msg1][msg4][msg7]...   │  ← czujniki 1-10
│ Partition 1: [msg2][msg5][msg8]...   │  ← czujniki 11-20
│ Partition 2: [msg3][msg6][msg9]...   │  ← czujniki 21-30
└──────────────────────────────────────┘
```

Pojęcie	Opis
Topic	Logiczny kanał danych (np. jeden temat = jeden typ eventu)
Partition	Podział topiku — umożliwia równoległe przetwarzanie
Offset	Numer porządkowy wiadomości w partycji — "zakładka w książce"
Producer	Aplikacja/urządzenie wysyłające dane do topiku
Consumer	Aplikacja odbierająca i przetwarzająca dane z topiku
Consumer Group	Grupa konsumentów dzieląca się pracą nad partycjami
Broker	Serwer Kafki przechowujący dane topiku

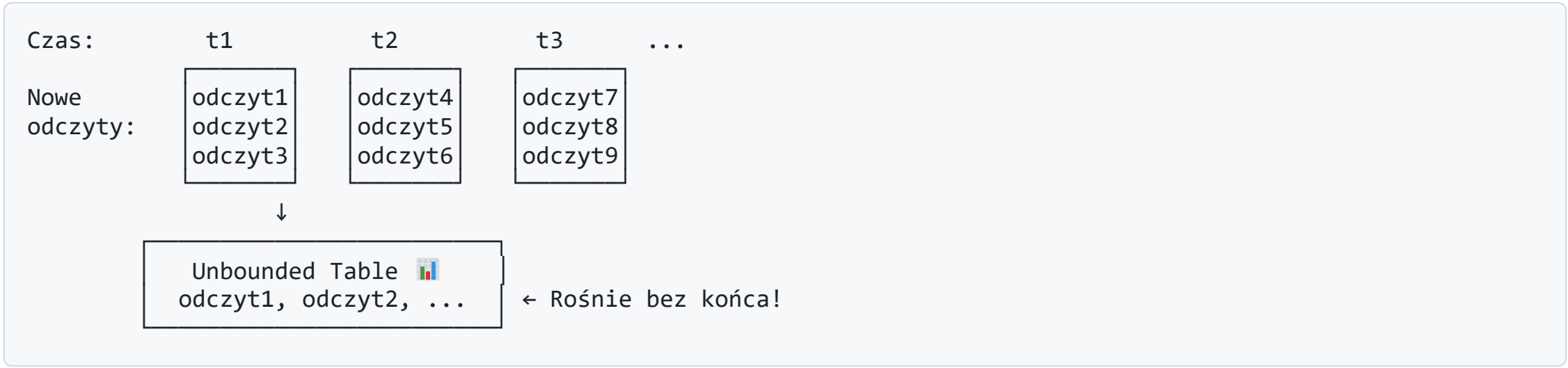
🏭 Dlaczego Kafka, a nie zwykła kolejka?

- 💾 **Trwałość** — wiadomości zapisywane na dysku, można je odtworzyć
- 🔁 **Wielu konsumentów** — ten sam topik może czytać Spark, dashboard i system alertów jednocześnie
- ⚡ **Wysoka przepustowość** — miliony wiadomości/sekundę w produkcyjnych wdrożeniach (LinkedIn, Netflix, Uber)
- 📈 **Skalowalność pozioma** — dodajesz partycje i brokery, gdy rośnie ruch

> 🎯 **Kafka + Spark Structured Streaming** = klasyczny duet w architekturach Big Data do przetwarzania danych w czasie rzeczywistym.

🏗️ Spark Structured Streaming — Unbounded Table

Kluczowa koncepcja: strumień danych = **nieskończona, rosnąca tabela**.



> 💡 Piszesz zwykłe zapytanie DataFrame/SQL — Spark automatycznie doklejba nowe wiersze i przelicza wynik!

💻 `readStream` / `writeStream` — podstawy API

Structured Streaming = **ten sam DataFrame API**, tylko `read` → `readStream`, `write` → `writeStream`:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName("JakoscPowietrza").getOrCreate()

# 1 Czytamy strumień (docelowo: Kafka; w Colab: pliki/rate source)
strumien = (spark.readStream
    .schema(schemat_czujnikow)
    .json("/dane/strumien_czujnikow/"))   # 👈 readStream zamiast read!

# 2 Transformacje — zwykły DataFrame API!
alerty = strumien.filter(col("pm25") > 100)

# 3 Zapis wyniku
query = (alerty.writeStream
    .format("console")
    .outputMode("append")
    .start())                              # 👈 writeStream zamiast write!
```

🔌 Docelowo w produkcji: czytanie z Kafki

```
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StringType, DoubleType, TimestampType

schemat = (StructType()
    .add("sensor_id", StringType())
    .add("pm25", DoubleType())
    .add("pm10", DoubleType())
    .add("temperatura", DoubleType())
    .add("wilgotnosc", DoubleType())
    .add("timestamp", TimestampType()))

# Czytamy z topiku Kafka 📥
raw = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "broker:9092")
    .option("subscribe", "powietrze-pm25")
    .option("startingOffsets", "latest")
    .load())

# Kafka dostarcza dane jako binary — dekodujemy JSON! 🔓
odczyty = (raw
    .select(from_json(col("value").cast("string"), schemat).alias("dane"))
    .select("dane.*"))
```

> 🎓 Na dzisiejszych ćwiczeniach nie mamy prawdziwego klastra Kafki w Colab — zasymulujemy strumień plikami JSON, ale kod docelowy z Kafką wygląda tak jak wyżej!

🪟 Windowed Aggregations — Okna czasowe

Chcemy: średnia PM2.5 **per czujnik**, w oknach **5-minutowych**.

```
from pyspark.sql.functions import window, avg, col

srednie_pm25 = (odczyty
    .groupBy(
        window(col("timestamp"), "5 minutes"),  # 👈 Tumbling window!
        col("sensor_id")
    )
    .agg(avg("pm25").alias("srednia_pm25")))

# Wynik:
# | window [10:00, 10:05) | sensor_id | srednia_pm25 |
# | window [10:05, 10:10) | sensor_id | srednia_pm25 |
```

> 💡 **Tumbling window** = okna stałej długości, bez nakładania się. Idealne do raportowania "co 5 minut".

🪟 Tumbling vs Sliding Windows

```
Tumbling (5 min):
|─────|─────|─────|─────|
 10:00 10:05 10:10 10:15

Sliding (5 min, slide 1 min):
|─────|
   |─────|
      |─────|
```

```
# Sliding window — świeższa średnia co minutę, z 5-minutowego okna 📐
srednia_krocząca = (odczyty
    .groupBy(
        window(col("timestamp"), "5 minutes", "1 minute"),  # 👈 slide!
        col("sensor_id")
    )
    .agg(avg("pm25").alias("srednia_pm25")))
```

> 🤔 **Think about it:** Chcesz wykrywać *trend wzrostowy* smogu szybciej niż co 5 minut. Jakie parametry okna wybierzesz?

Watermarking — spóźnione dane z czujników

Czujniki IoT bywają offline (słaby WiFi, awaria) — dane mogą dotrzeć **z opóźnieniem**.

```
from pyspark.sql.functions import window, avg, col

srednie_pm25 = (odczyty
    .withWatermark("timestamp", "10 minutes")   # 👈 akceptuj max 10 min opóźnienia
    .groupBy(
        window(col("timestamp"), "5 minutes"),
        col("sensor_id")
    )
    .agg(avg("pm25").alias("srednia_pm25")))

# Odczyt z timestampem 10:02, który dotrze o 10:11?
# 10:11 - 10:02 = 9 min < 10 min → AKCEPTOWANY ✅
# Odczyt z timestampem 10:01, który dotrze o 10:15?
# 10:15 - 10:01 = 14 min > 10 min → ODRZUCONY ❌
```

> ⚠️ **Bez watermarku** Spark trzyma cały stan agregacji w pamięci na zawsze → w końcu OOM!

🔄 Output Modes — jak zapisujemy wyniki?

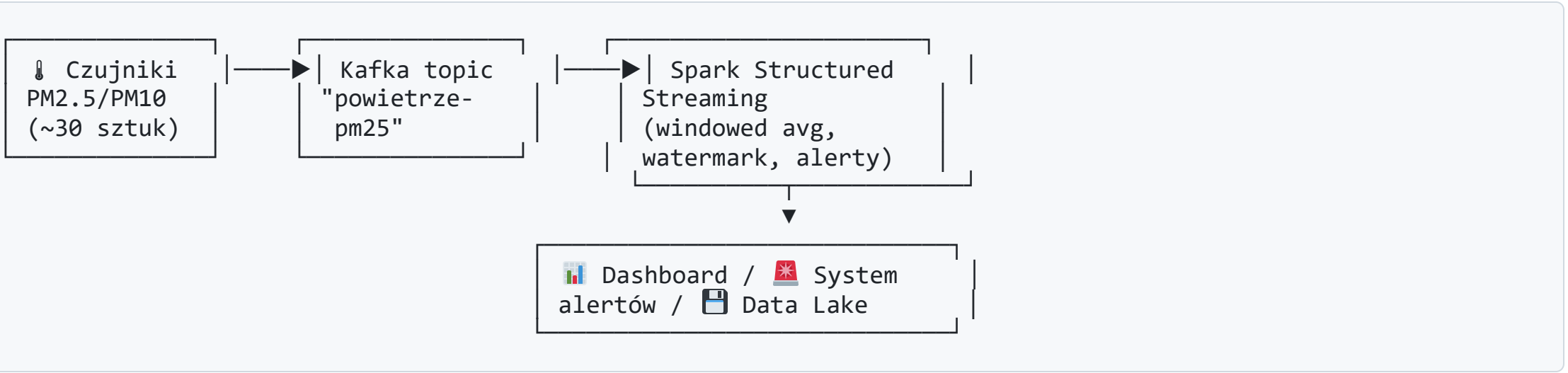
Output Mode	Co zapisuje?	Przykład użycia
Append	Tylko nowe wiersze	Surowe odczyty przekraczające próg PM2.5
Complete	Cała tabela wyników	Aktualna średnia PM2.5 per czujnik (groupBy)
Update	Tylko zmienione wiersze	Te same agregacje, ale mniej danych do zapisu

```
# Complete – pełna, zawsze aktualna tabela średnich 📊
query = srednie_pm25.writeStream.outputMode("complete").format("memory") \
    .queryName("srednie_pm25_tabela").start()

# Append – tylko nowe alerty (bez agregacji!) 🚨
query2 = alerty.writeStream.outputMode("append").format("console").start()
```

> ⚠️ **Pułapka:** `outputMode("append")` + `groupBy()` bez watermarku = błąd! Agregacje wymagają `"complete"` lub `"update"`.

🏗️ Architektura: Kafka + Spark dla monitoringu powietrza



> 💡 W Colab bez klastra Kafki: zastępujemy lewą stronę **generatorem plików JSON** — Spark i tak czyta je przez `readStream`, dokładnie tak samo jak z Kafki!

⚠️ Częste pułapki i best practices

Pułapka 🚨	Problem	Rozwiązanie ✅
Brak `checkpointLocation`	Utrata stanu przy restarcie	**ZAWSZE** ustawiaj checkpoint
Brak watermarku przy agregacjach	Rosnące zużycie pamięci → OOM	Dodaj `withWatermark()`
`append` + `groupBy` bez watermarku	Błąd wykonania	Użyj `"complete"` / `"update"` lub dodaj watermark
Zbyt małe partycje Kafki	Wąskie gardło przepustowości	Dostosuj liczbę partycji do równoległości
Testowanie od razu na Kafce	Złożony setup, trudny debug	Zacznij od file source / rate source (jak dziś!)
Ignorowanie `query.lastProgress`	Nie widzisz problemów wydajnościowych	Monitoruj regularnie w produkcji

🎓 Podsumowanie kursu — skąd dokąd doszliśmy!

Pamiętacie Wykład 1? 😏 Zaczynaliśmy od podstaw statystyki opisowej, a dziś kończymy na **streamingu w czasie rzeczywistym**!

Wykład	Temat
1	Statystyka opisowa, EDA i wnioskowanie
2	Formaty i przechowywanie danych (Parquet, Data Lake)
3–4	Big Data, Hadoop, Apache Spark (RDD)
5–6	Spark SQL i DataFrames
7–8	Feature engineering i modele ML
9–10	Spark MLlib i tuning hiperparametrów
11	Cloud Computing i GCP
12	**Structured Streaming i Kafka** 🌊

> 🎯 Od statystyki opisowej do przetwarzania strumieniowego w czasie rzeczywistym — przeszliście całą drogę inżyniera danych Big Data! 👏

Key Takeaways — Wykład 12

1. ✅ **Kafka** = rozproszony message broker: topics, partitions, producers, consumers, brokers
2. ✅ **Structured Streaming** = ten sam DataFrame API, z `readStream` / `writeStream`
3. ✅ Strumień danych = **unbounded table** — proste i intuicyjne podejście
4. ✅ **Windowed aggregations** (tumbling, sliding) agregują dane w czasie
5. ✅ **Watermarking** rozwiązuje problem spóźnionych danych z czujników
6. ✅ **Output modes** (append/update/complete) zależą od typu operacji
7. ✅ Kafka + Spark = standardowy stack produkcyjny do monitoringu w czasie rzeczywistym

📚 Polecane materiały

- 📖 Structured Streaming Programming Guide
- 📖 Spark + Kafka Integration Guide
- 📗 *Learning Spark, 2nd Edition* — rozdział o Structured Streaming
- 🎓 Confluent — Apache Kafka Fundamentals (darmowy kurs online)

🔮 Co dalej? — Egzamin i Projekt

- 📝 **Następne zajęcia (19.01.2027) — Egzamin teoretyczny** (70% oceny), obejmujący **Wykłady 1–12**. Sprawdźcie stronę **"Egzamin"**!
- 💼 **Projekt zaliczeniowy** (30% oceny) — termin oddania również zbliża się wielkimi krokami. Sprawdźcie stronę **"Projekt zaliczeniowy"**!
- 🏆 **Aktywność na zajęciach** = bonus +10% ponad 100%
- 💪 Dzisiejsze ćwiczenia + to zadanie domowe to ostatnia okazja by utrwalić materiał przed egzaminem!

> 🎯 To był długi, ale satysfakcjonujący semestr. Czas pokazać, co potraficie! 💪

❓ Pytania?

Dziękuję za świetny semestr! 🎉

Prowadzący: Pavlo Melnyk — melnyk@pjwstk.edu.pl